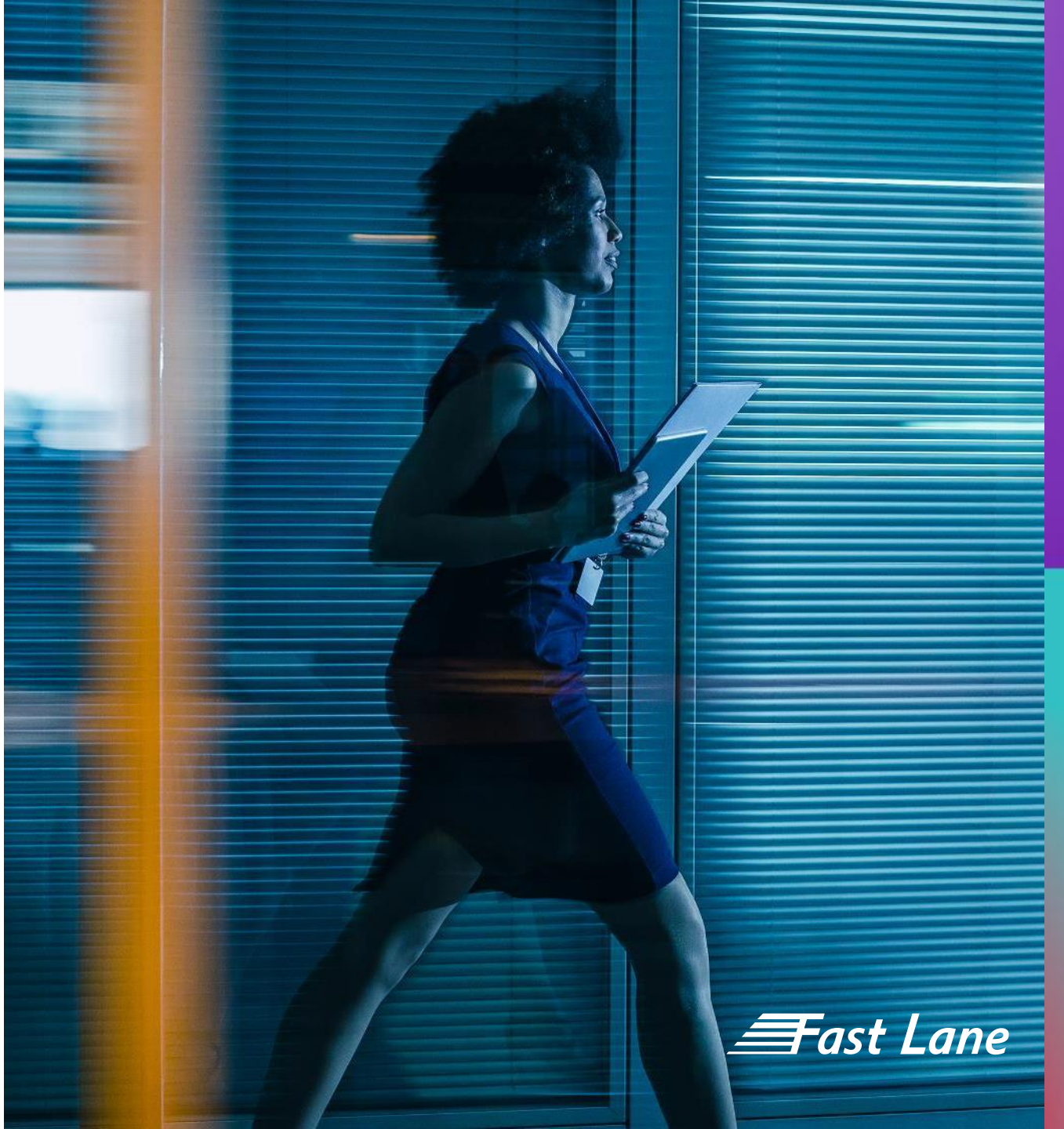
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 2

Building Distributed Apps with Aspire

Start, trace and test the Contoso system from one AppHost

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Aspire AppHost: Model the System

Resources, references, waits and parameters

02

ServiceDefaults: Telemetry, Resilience, Discovery

One call for OpenTelemetry, retries and health

03

Aspire Integrations: Hosting & Client

Hosting packages, client packages and emulators

04

Aspire Dashboard: Traces Across Services

Logs, traces, metrics and MCP for agents

05

Polyglot AppHost: Angular & Python

Vite and uvicorn apps beside .NET services

06

Aspire Testing: Distributed Integration Tests

Start the AppHost in a test, gate each step

Building distributed apps with Aspire

Start, trace and test the Contoso system from one AppHost

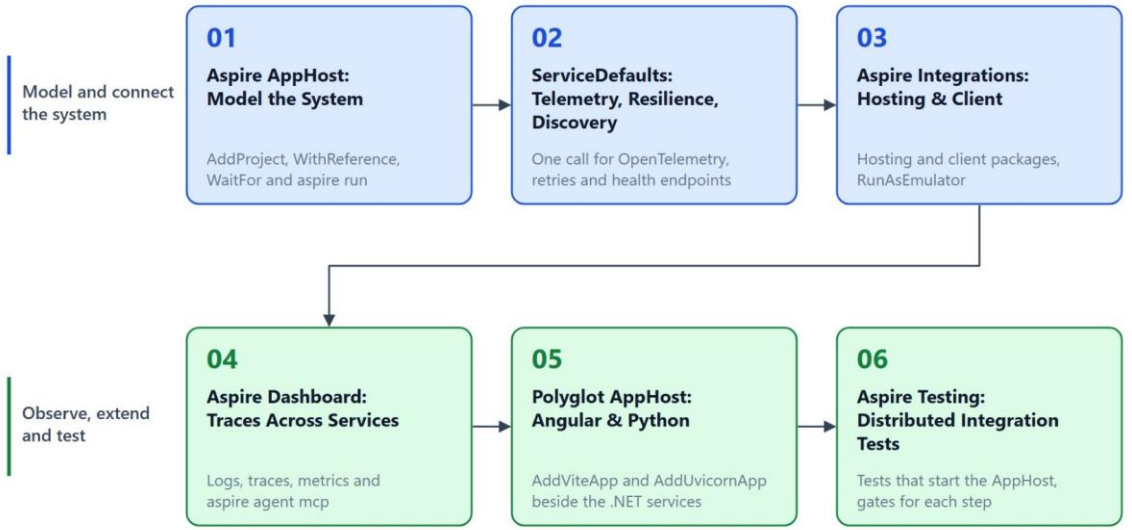
- **Model the system**: one AppHost starts every Contoso service and resource in order
- **Connect and observe**: shared defaults, emulated Azure resources and one dashboard
- **Extend and test**: Angular and Python apps in the same AppHost, tests that start it



Overview

From the AppHost model to distributed tests

- Model the system in an **AppHost** and start it with `aspire run`
- Give every service the same **ServiceDefaults** for telemetry, retries and discovery
- Connect Service Bus, Cosmos DB and Storage through **integrations** and emulators
- Follow one booking across services in the **dashboard**
- Host Angular and Python beside .NET, then **test** the event flow end to end



01

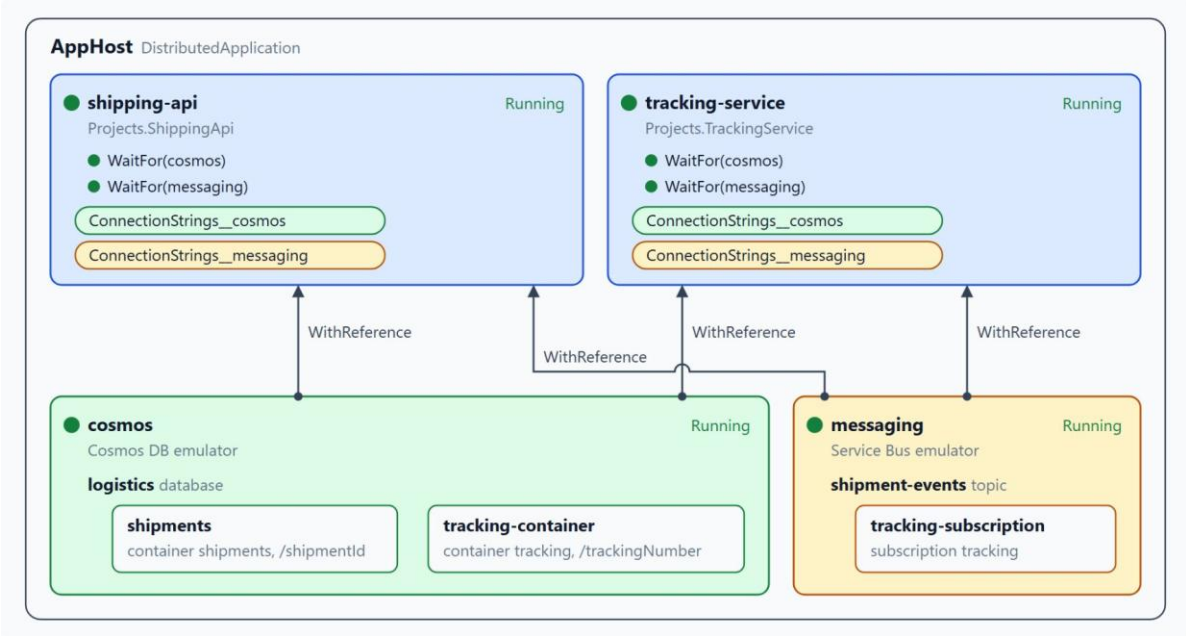
Aspire AppHost: Model the System

Resources, references, waits and parameters

Declaring and wiring resources

WithReference passes the connections, WaitFor orders the startup

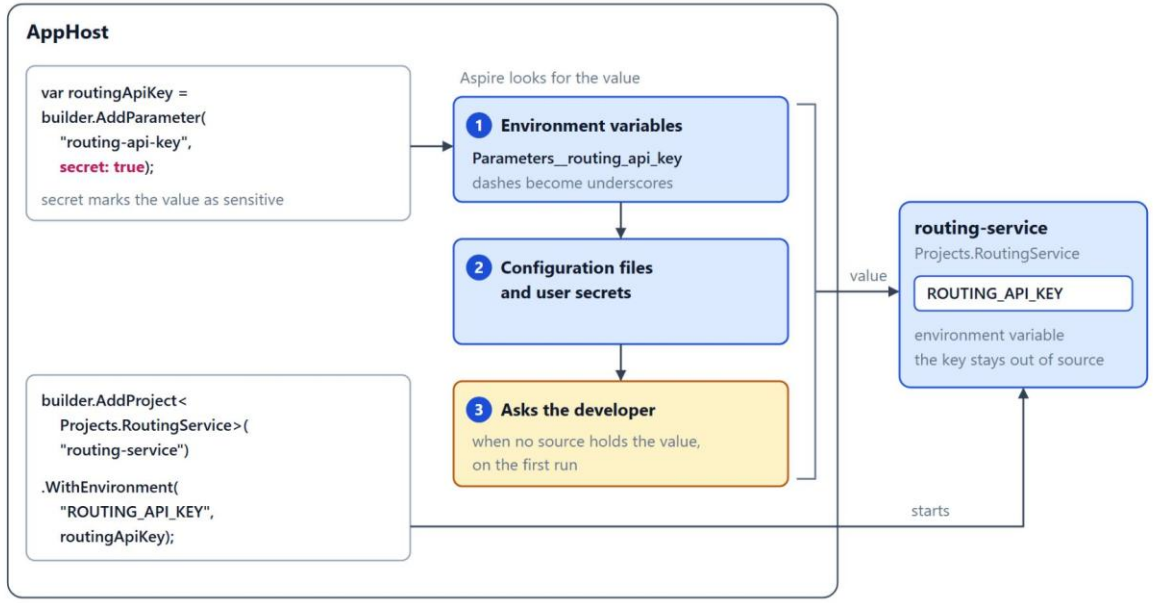
- Each builder call adds a **resource**; the generated `Projects` type catches a wrong reference at compile time
- `WithReference` hands a service the connection details as environment variables
- `WaitFor` holds a service until its dependency is Running; client retries still apply
- `aspire run` starts every resource and the dashboard; `--isolated` runs a second copy



Parameters for secrets and settings

AddParameter keeps the routing API key out of source

- `AddParameter` declares a value the AppHost needs but does not hard-code
- `secret: true` marks it as sensitive, such as the routing service's API key
- Aspire reads **environment variables** first, then configuration files and user secrets
- With no value anywhere, Aspire asks the developer on the first run



02

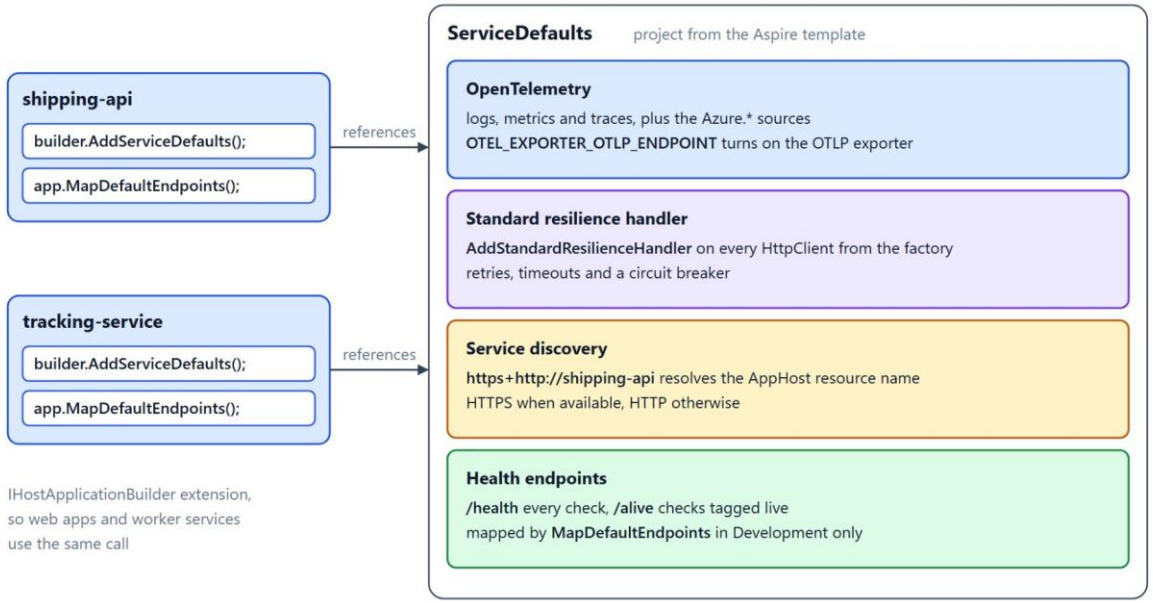
ServiceDefaults: Telemetry, Resilience, Discovery

One call for OpenTelemetry, retries and health

The ServiceDefaults project

One call gives a service telemetry, resilience, discovery and health checks

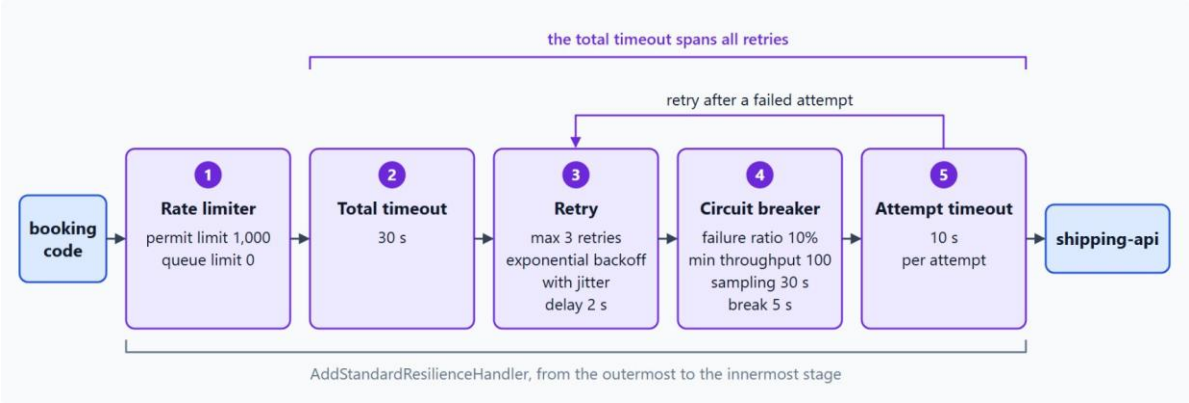
- Each service references ServiceDefaults and calls `AddServiceDefaults` once
- **OpenTelemetry** wires logs, metrics and traces; `OTEL_EXPORTER_OTLP_ENDPOINT` sets the target
- Every factory `HttpClient` gets the resilience handler and **service discovery**
- `MapDefaultEndpoints` maps `/health` and `/alive`, in Development only



The standard resilience handler

Five stages between the booking code and shipping-api

- `AddStandardResilienceHandler` adds retries, timeouts and a circuit breaker
- The **total timeout** of 30 s spans every retry; each attempt gets 10 s
- Retries replay requests, so the booking call to shipping-api must be **idempotent**
- Request clients through `IHttpClientFactory` or a typed client to get the handler



03

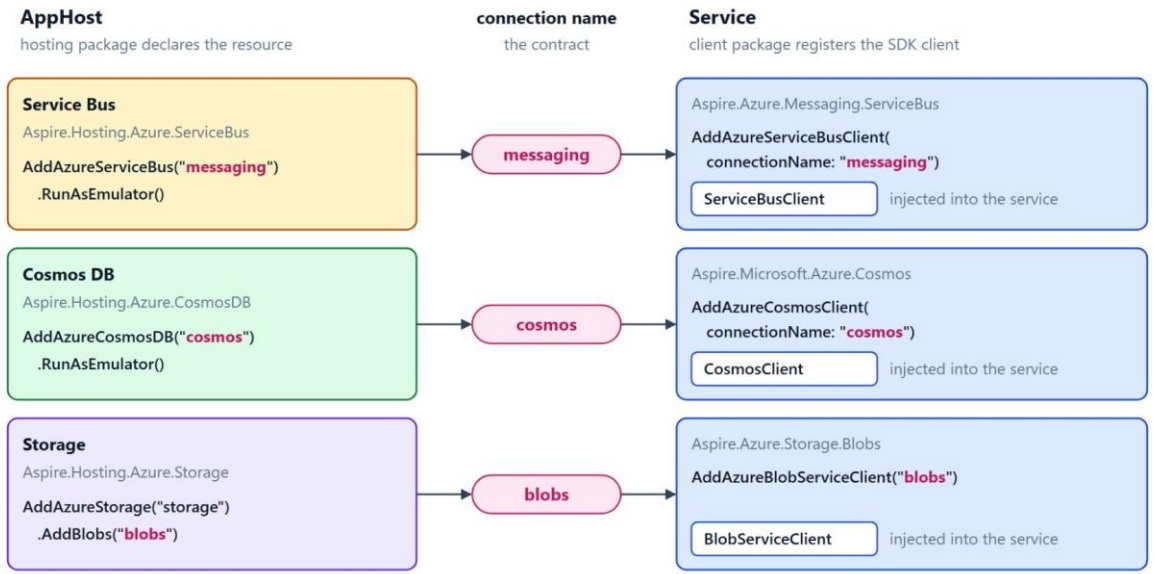
Aspire Integrations: Hosting & Client

Hosting packages, client packages and emulators

Hosting and client packages

The AppHost declares the resource, the service consumes the connection

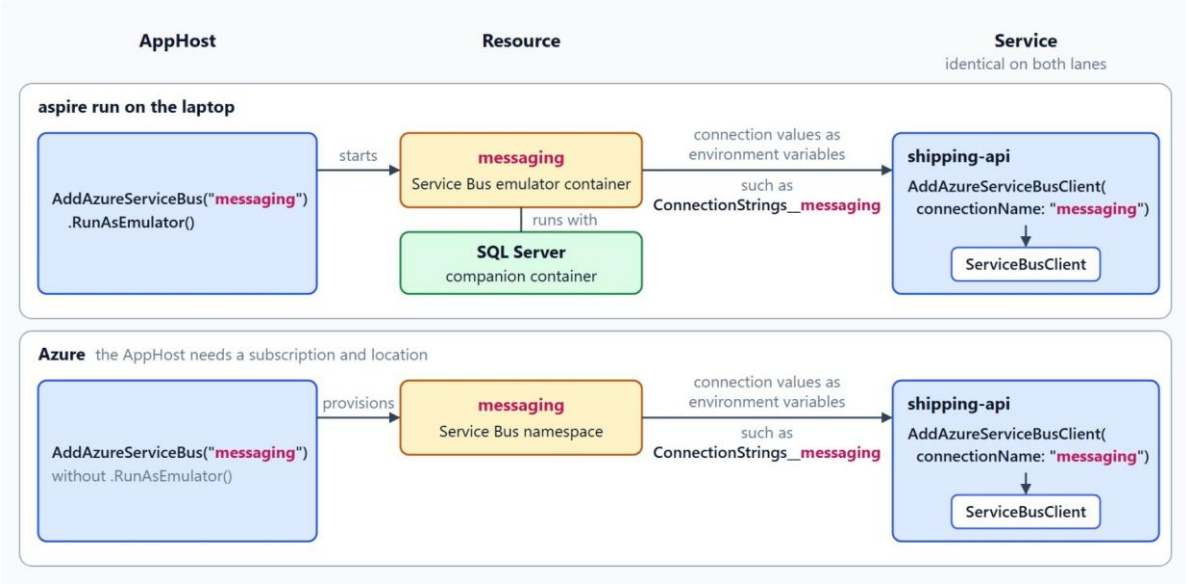
- A **hosting package** runs in the AppHost and adds calls such as `AddAzureServiceBus`
- `aspire add azure-service-bus` adds the package and records its version
- A **client package** registers the SDK client with its health checks and OpenTelemetry
- `connectionName` must match the AppHost resource name: the name is the contract



Local emulators and Azure deployment

The same messaging resource and client code on both lanes

- `RunAsEmulator` swaps the Azure resource for a local container during `aspire run`
- Service Bus runs with a SQL Server companion, Cosmos DB as vNext Linux emulator, Storage as **Azurite**
- Every emulator needs a **container runtime**; the first run pulls the images
- Without `RunAsEmulator`, Aspire provisions the real resource and the client code stays the same



04

Aspire Dashboard: Traces Across Services

Logs, traces, metrics and MCP for agents

Dashboard views

Resources, logs, traces and metrics in one browser tab

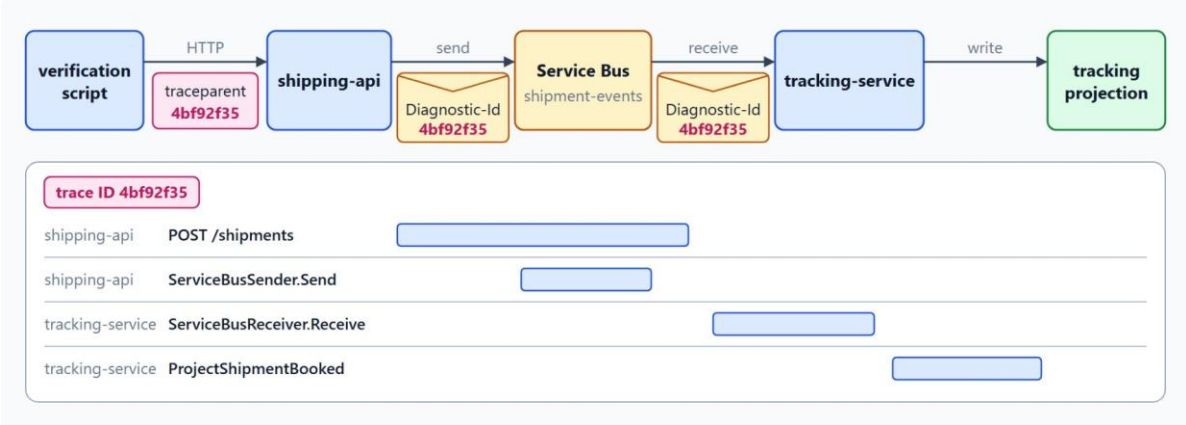
- **Resources** shows state and health, and restarts one resource without the rest
- **Console** and configuration show why a service failed to start
- **Structured logs** keep `TrackingNumber` as a field the dashboard filters on
- **Metrics** find the time window of a slowdown, a trace finds the request



Traces across HTTP and Service Bus

One trace ID follows the booking from shipping-api to tracking-service

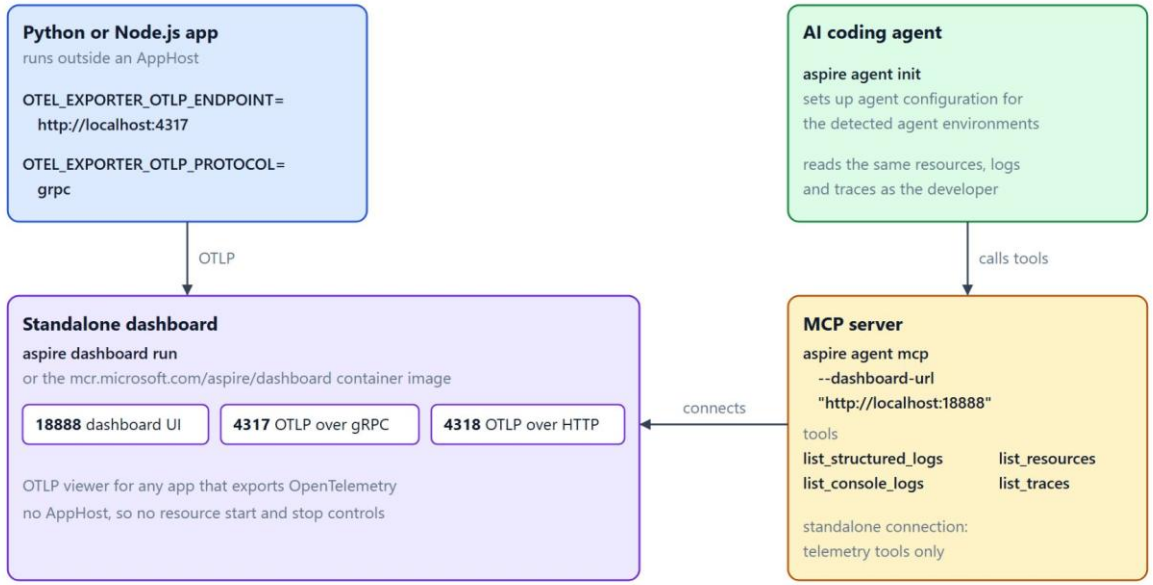
- HTTP calls carry the W3C trace context in the `traceparent` header
- Service Bus carries it in the `Diagnostic-Id` property of each message
- The waterfall adds a span per step; the **longest bar** names the slowest step
- The defaults switch on the Azure SDK spans and register the `Azure.*` source



MCP access and the standalone dashboard

Coding agents read the telemetry, any OTLP app can send it

- `aspire agent init` sets up agent configuration for the detected agent environments
- `aspire agent mcp` exposes telemetry as tools such as `list_traces` and `list_structured_logs`
- `aspire dashboard run` starts a **standalone dashboard** for apps outside an AppHost
- Apps send OTLP to port 4317 (gRPC) or 4318 (HTTP); the UI runs on 18888



05

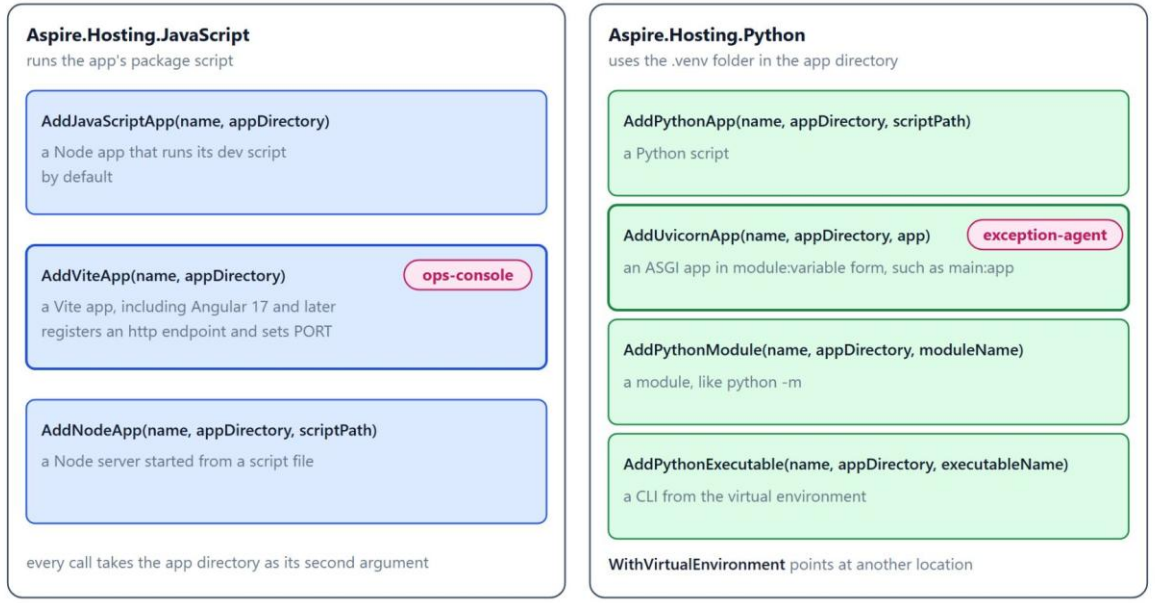
Polyglot AppHost: Angular & Python

Vite and uvicorn apps beside .NET services

Hosting calls for JavaScript and Python

The call follows how the app starts

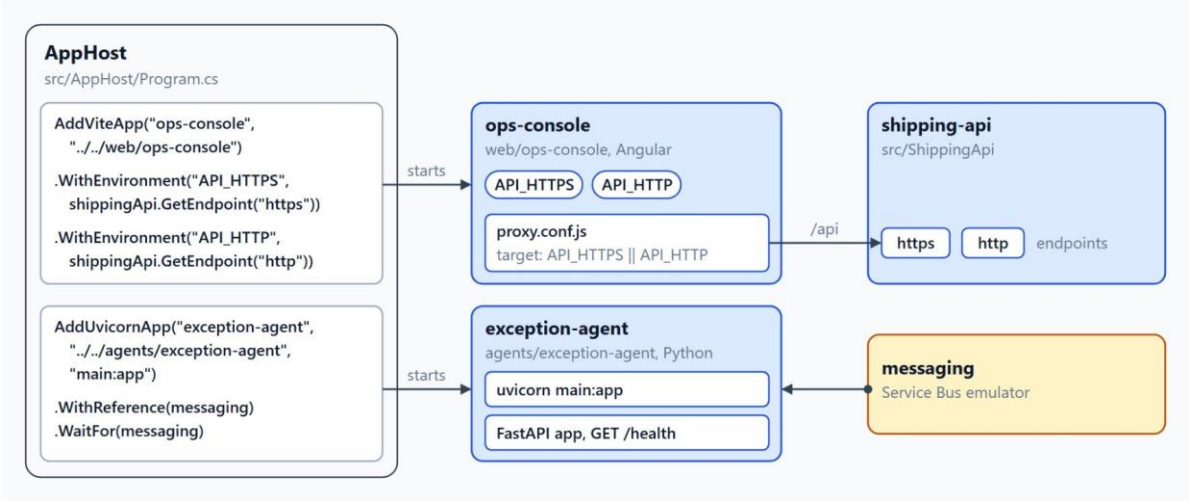
- `AddJavaScriptApp` runs a Node app's `dev` script, `AddNodeApp` a script file
- `AddViteApp` hosts Vite apps, including **Angular** 17 and later, and sets `PORT`
- `AddPythonApp` runs a worker script, `AddUvicornApp` an ASGI app such as a FastAPI agent
- Aspire uses the `.venv` in the app directory; `WithVirtualEnvironment` points elsewhere



Ops console and exception agent in the AppHost

One aspire run starts Angular and Python beside the .NET services

- `AddViteApp` starts `ops-console`; two `WithEnvironment` calls pass `API_HTTPS` and `API_HTTP`
- The Angular **proxy** forwards `/api` to shipping-api, so the browser needs no CORS setup
- `AddUvicornApp` runs the exception agent's `main:app` with a reference to `messaging`
- The dashboard lists each app beside an **installer** resource that reads Finished



06

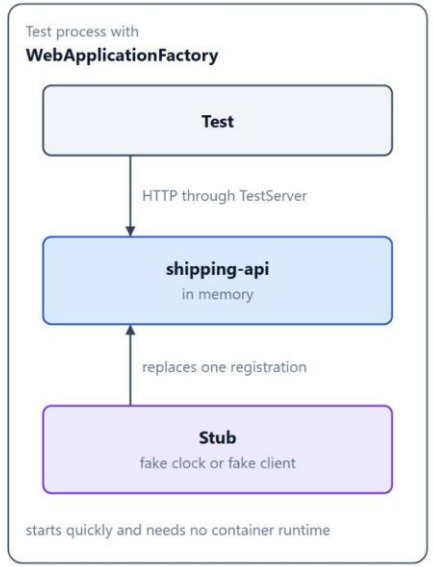
Aspire Testing: Distributed Integration Tests

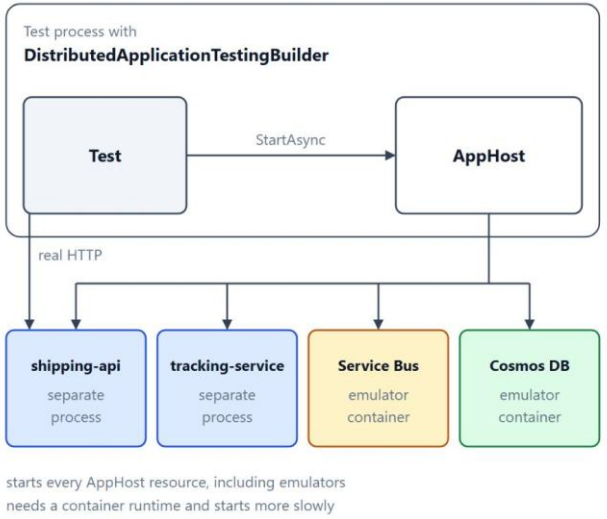
Start the AppHost in a test, gate each step

Testing the real AppHost

Aspire.Hosting.Testing starts every resource inside a test

- `DistributedApplicationTestingBuilder.CreateAsync` takes the AppHost type and starts it in the test
- `CreateHttpClient("shipping-api")` sends **real HTTP** to a real process
- `await using` stops the containers; a cancellation token caps the whole test
- `WebApplicationFactory` hosts one app in memory for logic inside a single service

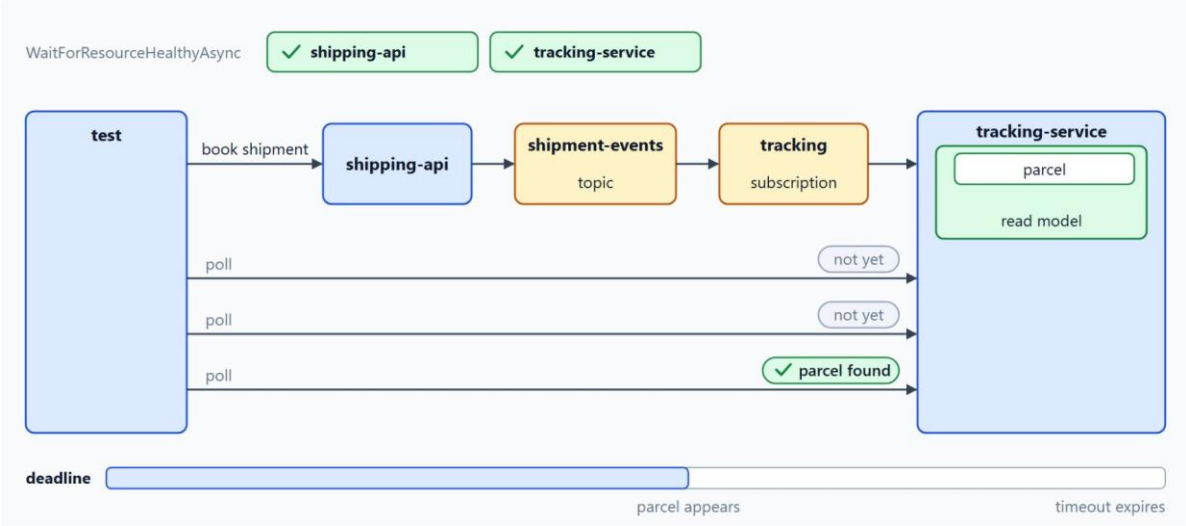




Testing the booking flow end to end

Wait for health, book a shipment, poll until the parcel appears

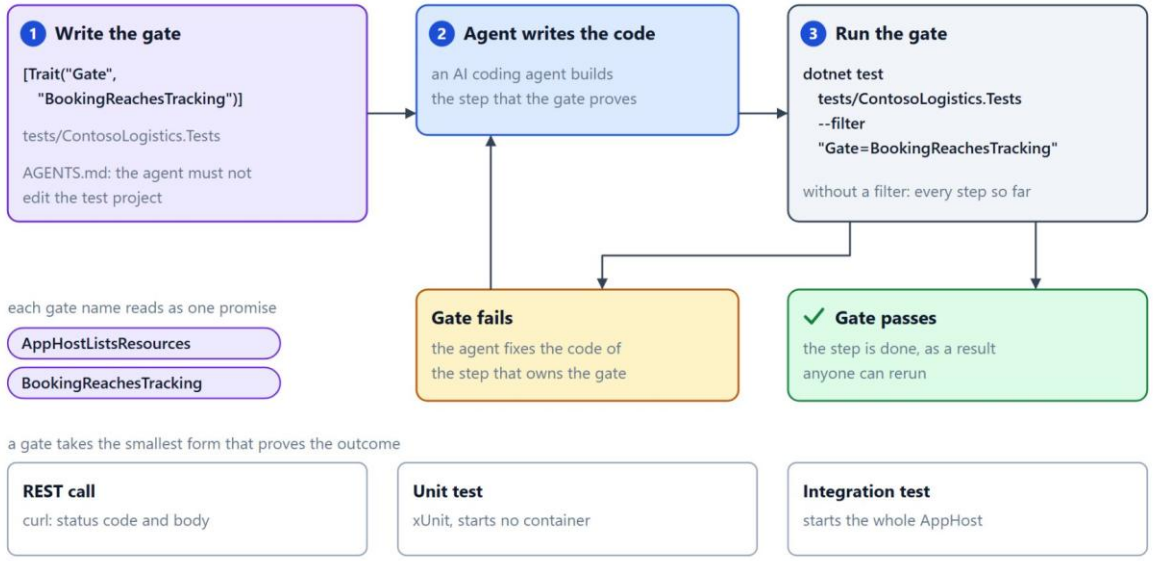
- `WaitForResourceHealthyAsync` waits for each resource the flow uses
- Health waits replace a guessed `Task.Delay` startup time
- The test books through shipping-api, then **polls** tracking-service with a deadline
- One assertion per flow stage, so a failure names the stage that broke



Gate tests

A fixed test proves each step that an agent builds

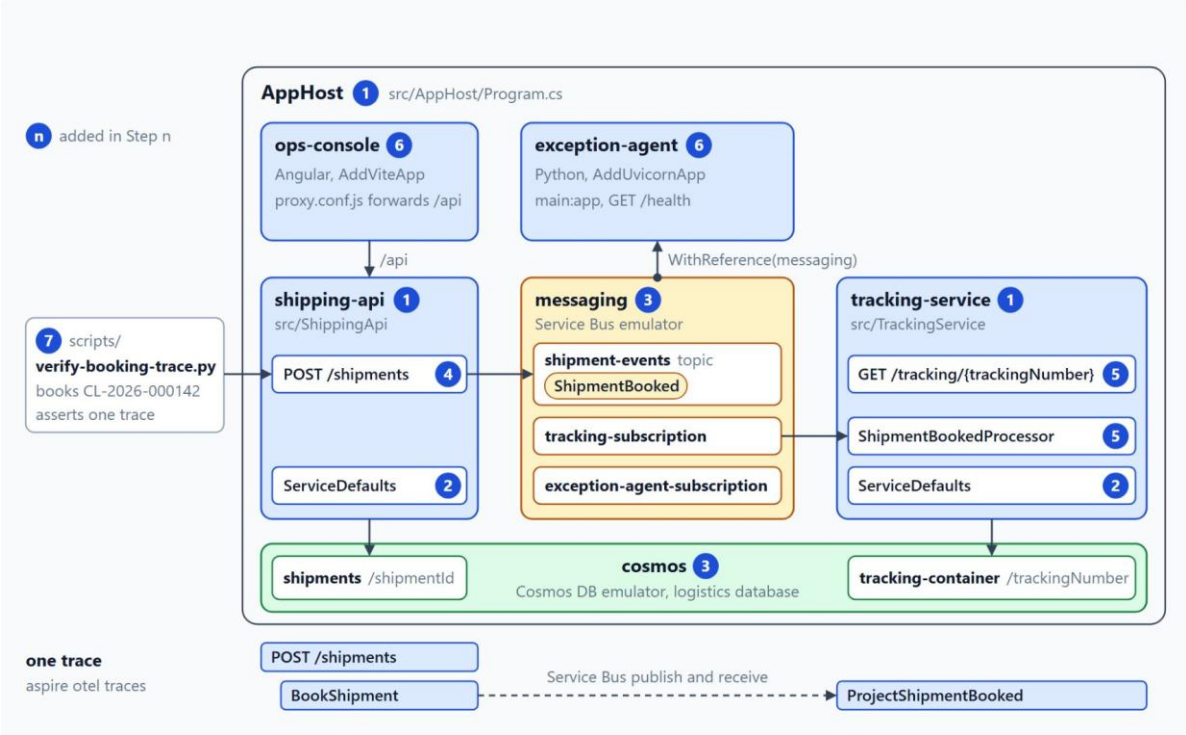
- A gate proves one step is done, checked from outside the code the step changes
- Write the gate first; `AGENTS.md` tells the agent to leave the test project unchanged
- `[Trait("Gate", ...)]` lets `dotnet test --filter` run exactly one step's gate
- A gate takes the smallest form: a `curl` call, a unit test or a full AppHost test



What the lab builds

Seven steps from design documents to one traced booking

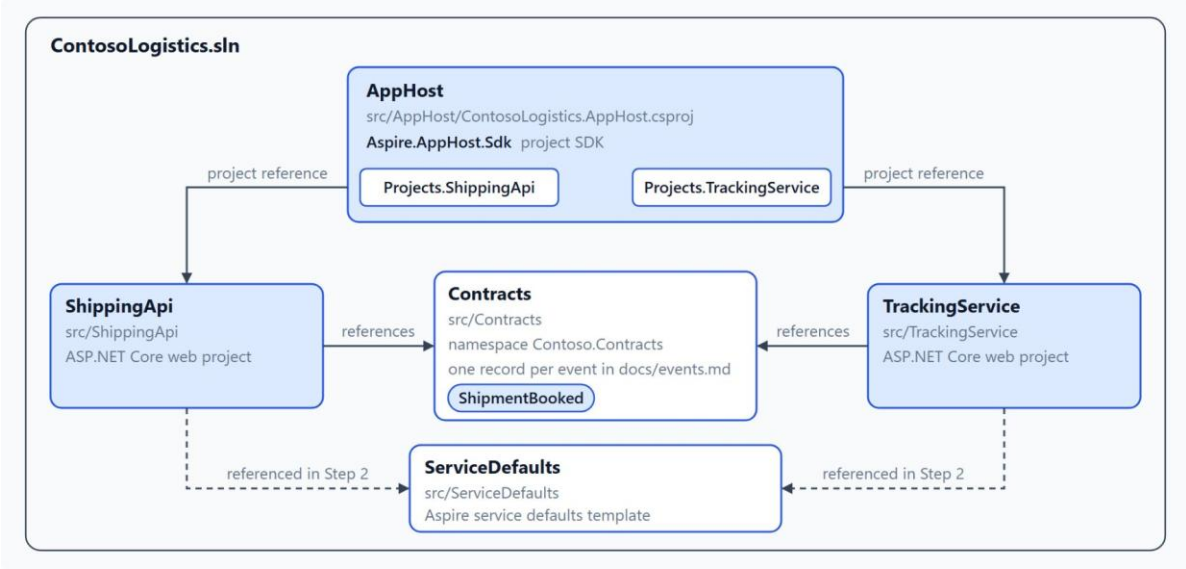
- Start from the lab 1 design documents, with no service running
- One `aspire start` runs both emulators, shipping-api, tracking-service, ops-console and exception-agent
- A booking travels through **Service Bus** from shipping-api to tracking-service
- `verify-booking-trace.py` asserts **one trace** across both services



Solution projects

Step 1 shapes ContosoLogistics.sln from the design

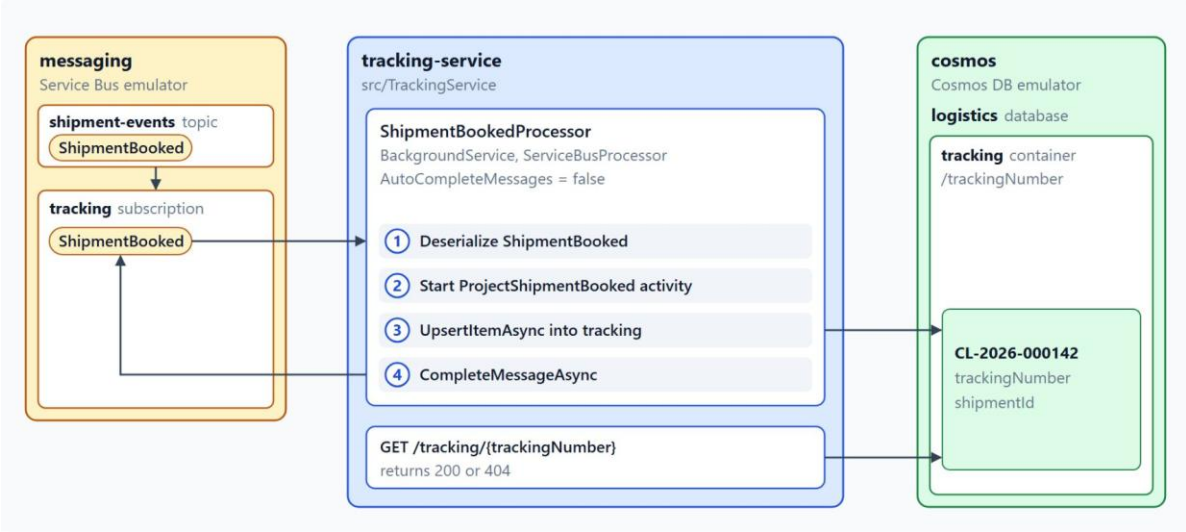
- The agent reads `docs/architecture.md` and `docs/events.md` and proposes the projects first
- Five projects: AppHost, ServiceDefaults, Contracts, ShippingApi and TrackingService
- AppHost references become `Projects.ShippingApi` and `Projects.TrackingService`
- **Contracts** holds one record per event, such as `ShipmentBooked`



Projecting the booking in tracking-service

Step 5 writes the tracking document before it completes the message

- `ShipmentBookedProcessor` reads `ShipmentBooked` from the `tracking` subscription
- It upserts the tracking document in Cosmos DB, then **completes** the message
- `GET /tracking/{trackingNumber}` returns 200 with the document, or 404
- The `BookingReachesTracking` **gate** is written before the code and passes after it



Takeaways

What to carry into the hosting module

- Describe the system once in the **AppHost**; one command starts it in order
- Opt every service into **ServiceDefaults** with one call
- Keep `connectionName` equal to the resource name; emulators run the same code
- Follow one booking as one **trace** across HTTP and Service Bus
- Write **gates** before the agent writes code, and wait for health in every test

